



國立中山大學資訊工程學系

碩士論文

Department of Computer Science and Engineering

National Sun Yat-sen University

Master's Thesis

最多與最長共同遞增區間子序列問題

The Most and Longest Common Increasing Interval

Subsequence Problems

研究生：陳冠傑

Guan-Jie Chen

指導教授：楊昌彪 教授

Prof. Chang-Biau Yang

中華民國 113 年 8 月

August 2024

The Most and Longest Common Increasing Interval Subsequence Problems

A Thesis

Submitted to the Faculty

of

Department of Computer Science and Engineering

National Sun Yat-sen University

by

Guan-Jie Chen

In Partial Fulfillment of the

Requirements for the Degree

of

Master of Science

August 2024

國立中山大學研究生學位論文審定書

本校資訊工程學系碩士班

研究生陳冠傑（學號：M113040073）所提論文

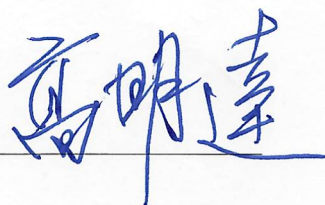
最多與最長共同遞增區間子序列問題

The Most and Longest Common Increasing Interval Subsequence Problems

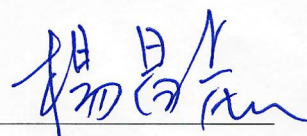
於中華民國 113 年 8 月 13 日經本委員會審查並舉行口試，符合
碩士學位論文標準。

學位考試委員簽章：

召集人 高明達



委員 楊昌彪



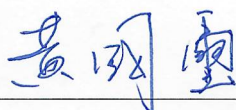
委員 謝育平



委員 李淑敏

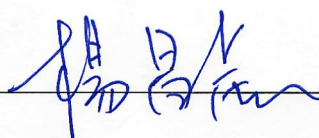


委員 黃國璽



委員

指導教授(楊昌彪)



(簽名)

The Most and Longest Common Increasing Interval Subsequence Problems

**by
Guan-Jie Chen**

A thesis submitted to the Graduate Division in partial
fulfillment of the requirements for the degree of
Master of Science
in the Department of Computer Science and Engineering
National Sun Yat-sen University
Kaohsiung, Taiwan, Republic of China
August, 2024

Approved by :

Convenor :
Professor

Ming-Tat Ko

Member :
Professor

CByang

Member :
Professor

Yuh-Pyng Shieh

Member :
Professor

Shu-Li

Member :
Professor

Huo-Si Huang

Advisor :

CByang

謝 辭

首先，我要感謝我的指導教授楊昌彪教授。感謝您在我整個研究過程中的指導與支持，您的豐富經驗和悉心教導，讓我在學術上得以不斷進步，完成這篇論文。同時，我也要感謝我的口試委員高明達博士、謝育平教授、李淑敏教授。在我最需要指導的時候，您們給予了我許多寶貴的意見，幫助我完善這篇論文。

特別感謝學長國璽，感謝您在百忙之中協助我修改論文，您的建議與指導對我的論文品質有著極大的提升。同時，我也要感謝我的學長姐們：健廷、志軒、映妤、承翰、冠廷、宗霖。感謝你們在學業和生活上的幫助與支持，讓我在學習的道路上不再孤單。感謝我的同學：鈺宸、之承、宥諺、韻安、玟雯。你們的陪伴與鼓勵，使我能夠在研究過程中保持積極的心態，共同度過了許多挑戰和困難。我的學弟妹們：適宏、鑫昌、景翔、彥宇、紋瑜、宛妮，感謝你們的支持與關心，讓我在研究的路上感受到滿滿的動力。

最後，我要感謝我的家人。感謝你們一直以來的理解和支持，是你們的鼓勵和關愛，讓我能夠專心完成這篇論文。

再次感謝所有在這段旅程中給予我幫助和支持的人，沒有你們的陪伴和支持，我無法完成這篇論文。

陳冠傑 2024 年 8 月

於 西子灣



摘要

最長共同遞增子序列 (LCIS) 問題是在兩個給定的序列中找到長度最長的共同遞增子序列。在本論文中，我們基於 LCIS 問題提出了兩個新問題，分別是最多共同遞增區間子序列 (MCIIS) 問題和最長共同遞增區間子序列 (LCIIS) 問題。這兩個問題通過將單一數值替換為區間來擴展 LCIS 問題。MCIIS 問題的目標是找到兩個區間序列中形成的最多個最大共同區間，這些區間可以組成一個區間數量最多的遞增區間序列。而 LCIIS 問題則是要找到兩個區間序列中形成的最大共同區間，這些區間可以構成聯集長度最大的遞增區間序列。我們提出了解決 MCIIS 和 LCIIS 問題的演算法，其時間複雜度分別是 $O(n^4)$ 和 $O(n^3)$ ，其中 n 代表輸入區間序列中的區間數量。

關鍵詞：最長共同遞增子序列、最多共同遞增區間子序列、最長共同遞增區間子序列、區間、區間序列

ABSTRACT

The longest common increasing subsequence (LCIS) problem is to find the common increasing subsequence with the maximum length between two given sequences. In this thesis, we define two new problems derived from the LCIS problem, the most common increasing interval subsequence (MCIIS) problem and the longest common increasing interval subsequence (LCIIS) problem. These two problems extend the LCIS problem by replacing single values with intervals. The MCIIS problem aims to identify the maximum common intervals formed between two interval sequences, where these intervals can build an increasing interval sequence with the maximum quantity of intervals. The LCIIS problem seeks to identify the maximum common intervals formed between two interval sequences, where these intervals can construct an increasing interval sequence whose union length of intervals is maximized. We propose algorithms for solving the MCIIS and the LCIIS problems in $O(n^4)$ and $O(n^3)$ time, respectively, where n denotes the number of intervals in the input interval sequences.

Keywords: longest common increasing subsequence, most common increasing interval subsequence, longest common increasing interval subsequence, interval, interval sequence

TABLE OF CONTENTS

	Page
THESIS VERIFICATION LETTER	i
ACKNOWLEDGMENTS	iii
CHINESE ABSTRACT	iv
ABSTRACT	v
TABLE OF FIGURES	viii
TABLE OF TABLES	ix
Chapter 1. Introduction	1
Chapter 2. Preliminaries	6
2.1 Longest Common Increasing Subsequence	6
2.2 Most Increasing Interval Subsequence	7
2.3 Longest Increasing Interval Subsequence	9
2.4 Most Common Increasing Interval Subsequence	11
2.5 Longest Common Increasing Interval Subsequence	13
Chapter 3. Our Algorithms	15
3.1 Most Common Increasing Interval Subsequence	15
3.1.1 The Algorithms	18
3.1.2 Complexity Analysis	23
3.2 Longest Common Increasing Interval Subsequence	23

	Page
3.2.1 The Algorithms	29
3.2.2 Complexity Analysis	32
Chapter 4. Experimental Results	35
4.1 Most Common Increasing Interval Subsequence	35
4.2 Longest Common Increasing Interval Subsequence	37
Chapter 5. Conclusion	40
BIBLIOGRAPHY	41

TABLE OF FIGURES

Figure	Page
1.1 Extension flow of the problems.	4
2.1 An example of a maximal common interval.	11
3.1 An example of the interval combination.	24

TABLE OF TABLES

Table		Page
1.1	The studies on LIS and LCIS problems. m and n : the lengths of the sequences A and B ; l : the length of the answer; Σ : the alphabet set; Sort: the time required to sort each input sequence.	3
1.2	A summary of the MIIS and the LIIS problems [5], as well as the proposed MCIIS and LCIS problems. m and n : the numbers of intervals in the interval sequences X and Y , respectively, where $m \leq n$	4
2.1	An example of the LCIS algorithm proposed by Yang <i>et al.</i> [20], where input sequences $A = \langle 2, 1, 5, 3, 8 \rangle$ and $B = \langle 1, 3, 5, 4, 8, 2 \rangle$. The LCIS answer is either $\langle 1, 5, 8 \rangle$ or $\langle 1, 3, 8 \rangle$, both of length 3.	7
2.2	An example of the MIIS algorithm (comparison by full interval) proposed by Chen and Yang [5], where the given interval sequence $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$. The MIIS answer can be $\langle [1, 3], [4, 4], [5, 6], [10, 10] \rangle$ or $\langle [2, 3], [4, 4], [5, 6], [10, 10] \rangle$, containing four intervals.	9
2.3	An example of the LIIS algorithm (comparison by full interval) proposed by Chen and Yang [5], where the given interval sequence $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$. The answer is $\langle [1, 3], [7, 8], [9, 11] \rangle$ with interval sequence length 8.	11

Table	Page
2.4 An aligned example of the MCIIS problem, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The MCIIS answer is $\langle [3, 6], [5, 7], [10, 11] \rangle$ with three intervals.	13
2.5 An aligned example of the LCIIS problem, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The LCIIS answer is $\langle [3, 6], [4, 12] \rangle$	14
3.1 An example of finding MCIIS, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The MCIIS answer is $\langle [3, 6], [5, 7], [10, 11] \rangle$, consisting of 3 intervals.	18
3.2 An example of finding LCIIS by using the 3-tuple entry $\langle l, c^s, c^e \rangle$, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The answer is $\langle [3, 6], [4, 12] \rangle$, with a total length of 10.	25
3.3 An example of finding LCIIS by using the 2-tuple entry $\langle l, c^e \rangle$, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The answer is $\langle [3, 6], [4, 12] \rangle$, with a total length of 10.	29
4.1 The number of entries in a cell and the execution time for the MCIIS algorithm, with non-duplicate starting and ending values. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$	36
4.2 The number of entries in a cell and the execution time for the MCIIS algorithm, with random starting and ending values. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$	37

4.3	The number of entries in a cell and the execution time for the LCIS algorithm, with $\alpha = 10$. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$	38
4.4	The number of entries in a cell and the execution time for the LCIS algorithm, with $\alpha = 100$. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$	39

CHAPTER 1

Introduction

A sequence, comprised of a series of data elements arranged in a specific order, is commonly found in a variety of applications. Sequences have been extensively studied in various fields. For instance, DNA sequences enable the determination of species characteristics and behaviors. When exploring evolutionary relationships between species, comparing the similarity of DNA sequences serves as a valuable tool, such as FASTA [16] and BLAST [2]. In addition to character sequences like DNA or protein sequences, numeric sequences play a crucial role in our lives, including time series [9, 19], stock prices [10], and data compression [6, 7, 11]. To extract additional insights from numeric sequences, many related problems have been proposed and investigated [5, 12, 15, 17, 20].

The *longest increasing subsequence* (LIS) problem [17] aims to find a strictly increasing subsequence with maximum length in a given sequence. For example, given a sequence $A = \langle 7, 1, 5, 8, 6, 9 \rangle$, its LIS length is 4, where the LIS solution is $\langle 1, 5, 8, 9 \rangle$ or $\langle 1, 5, 6, 9 \rangle$.

The LIS problem was initially defined by Schensted [17] in 1961. For a given sequence of length n , Schensted [17] proposed an algorithm with a time complexity of $O(n \log n)$ by utilizing Young tableau and binary search techniques. Subsequently, Hunt and Szymanski [12] introduced an algorithm that leveraged matching pairs with the van Emde Boas tree [18], achieving an improved time complexity of $O(n \log \log n)$ when the input sequence is a permutation of $\{1, 2, \dots, n\}$. Following this, Bespamyatnikh and Segal [4] devised a method to enumerate all LIS solutions with the same time complexity of $O(n \log \log n)$. Crochemore and Porat [8] further reduced the time complexity to $O(n \log \log l)$, where l represents the LIS length.

Lastly, Alam and Rahman [1] proposed a divide-and-conquer algorithm with a time complexity of $O(n \log n)$.

The *longest common increasing subsequence* (LCIS) problem focuses on identifying a longest common subsequence that exhibits increasing values from two given sequences. For example, consider two sequences $A = \langle 2, 1, 5, 3, 8 \rangle$ and $B = \langle 1, 3, 5, 4, 8, 2 \rangle$. In this case, the LCIS solution can be either $\langle 1, 5, 8 \rangle$ or $\langle 1, 3, 8 \rangle$, both of length 3.

The LCIS problem was first proposed by Yang *et al.* [20] in 2005, who devised an algorithm that can find a solution in $O(mn)$ time, where m and n represent the lengths of the two input sequences. Subsequently, Kutz *et al.* [13] introduced an algorithm that achieves a solution in $O((m + nl) \log \log |\Sigma| + \text{Sort})$ time, where l represents the LCIS length, $|\Sigma|$ denotes the alphabet size, and Sort represents the time required to sort each input sequence. Additionally, Lo *et al.* [15] designed a diagonal-based algorithm, which can find a solution in $O((n + l(m - l)) \log \log |\Sigma|)$ time. The related studies on the LIS problem and the LCIS problem are summarized in Table 1.1.

The *most increasing interval subsequence* (MIIS) problem and the *longest increasing interval subsequence* (LIIS) problem, proposed by Chen and Yang [5], are variations of the LIS problem where each element in the original LIS problem is replaced by an interval. The MIIS problem aims to find an increasing interval subsequence that has the maximum quantity of intervals, while the LIIS problem focuses on identifying an increasing interval subsequence whose union length of intervals is maximized. For example, given an interval sequence $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$, the solution to the MIIS problem is $\langle [1, 3], [4, 4], [5, 6], [10, 10] \rangle$ or $\langle [2, 3], [4, 4], [5, 6], [10, 10] \rangle$ containing four intervals (comparison by full interval), and the solution to the LIIS problem is $\langle [1, 3], [7, 8], [9, 11] \rangle$ with a total length of 8 (comparison by full interval). The related studies on the MIIS problem and the LIIS problem are summarized in Table 1.2.

Table 1.1: The studies on LIS and LCIS problems. m and n : the lengths of the sequences A and B ; l : the length of the answer; Σ : the alphabet set; Sort: the time required to sort each input sequence.

The longest increasing subsequence (LIS) problem [5]			
Year	Author(s)	Time complexity	Note
1961	Schensted [17]	$O(n \log n)$	Young tableau, binary search
1977	Hunt and Szymanski [12]	$O(n \log \log n)$	match pair, van Emde Boas tree
2000	Bespamyatnikh and Segal [4]	$O(n \log \log n)$	exhaustive
2010	Crochemore and Porat [8]	$O(n \log \log l)$	split blocks
2013	Alam and Rahman [1]	$O(n \log n)$	divide-and-conquer

The longest common increasing subsequence (LCIS) problem			
Year	Author(s)	Time complexity	Note
2005	Yang <i>et al.</i> [20]	$O(mn)$	dynamic programming
2011	Kutz <i>et al.</i> [13]	$O((m + nl) \log \log \Sigma + \text{Sort})$	van Emde Boas tree
2020	Lo <i>et al.</i> [15]	$O((n + l(m - l)) \log \log \Sigma)$	diagonal

Similar to the relationship between the LIS problem [17] and the MIIS problem (as well as the LIIS problem) [5], this thesis introduces the *most common increasing interval subsequence* (MCIIS) problem and the *longest common increasing interval subsequence* (LCIIS) problem as the interval versions of the LCIS problem [20]. Figure 1.1 shows the extension flow of the problems. Then, we devise algorithms for solving the MCIIS and LCIIS problems based on the dynamic programming (DP) approach [3, 14]. By leveraging specialized data structures and the properties of relationships between intervals, we can significantly reduce the required computational time and space. Our algorithms can solve the MCIIS problem in $O(n^4)$ time and space, and the LCIIS problem in $O(n^3)$ time and space. Additionally, through space reuse, we further reduce the space complexity to $O(n^3)$ for the MCIIS problem and $O(n^2)$ for the LCIIS problem.

Table 1.2: A summary of the MIIS and the LIIS problems [5], as well as the proposed MCIIS and LCIIS problems. m and n : the numbers of intervals in the interval sequences X and Y , respectively, where $m \leq n$.

Year	Author(s)	Time complexity	Note
The most increasing interval subsequence (MIIS) problem			
2024	Chen and Yang [5]	$O(n \log n)$ $O(n \log^2 n)$	one interval sequence, greedy, binary search, domination
The longest increasing interval subsequence (LIIS) problem			
2024	Chen and Yang [5]	$O(n \log n)$	one interval sequence, greedy, binary search, domination
The most common increasing interval subsequence (MCIIS) problem			
2024	This thesis	$O(n^4)$	two interval sequences, domination, dynamic programming
The longest common increasing interval subsequence (LCIIS) problem			
2024	This thesis	$O(n^3)$	two interval sequences, domination, dynamic programming

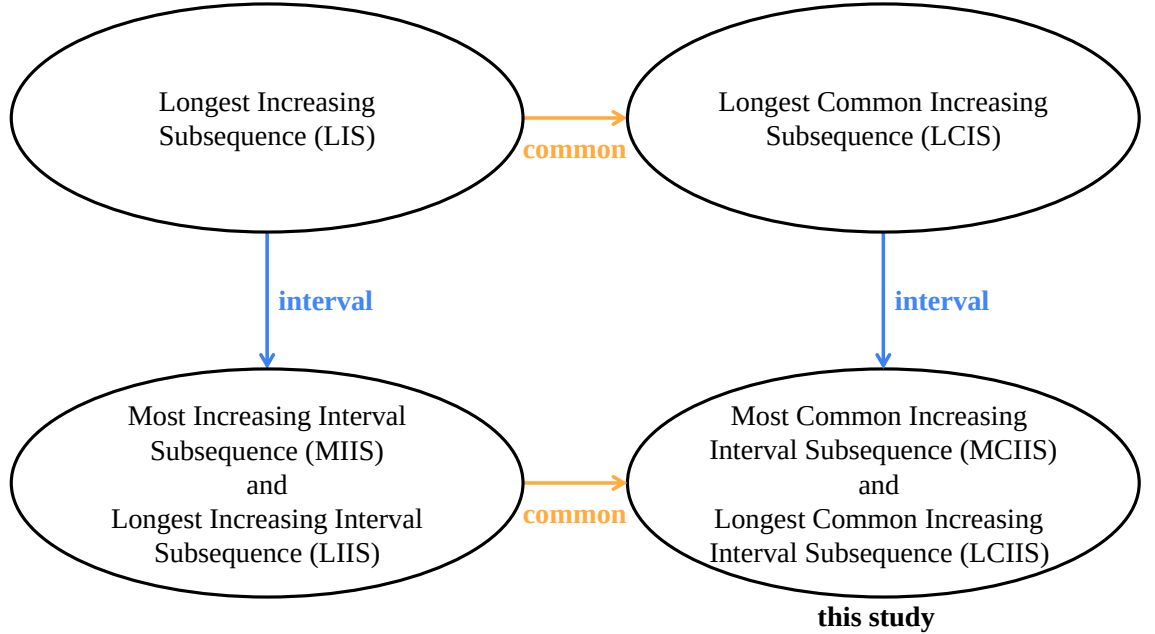


Figure 1.1: Extension flow of the problems.

The rest of this thesis is organized as follows. In Chapter 2, we review the preliminaries of the LCIS, MIIS and LIIS problems, and define the MCIIS and LCIIS problems. We present our algorithms for solving the proposed problems in Chapter 3. Chapter 4 shows experimental results compared with previous algorithms. Finally, in Chapter 5, we present our conclusions and outline future research directions.

CHAPTER 2

Preliminaries

The MCIIS problem and the LCIS problem are variations of the LCIS problem. This chapter first introduces the LCIS problem, the MIIS problem and the LIIS problem, followed by the definitions of the MCIIS problem and the LCIS problem.

2.1 Longest Common Increasing Subsequence

Given two sequences $A = \langle a_1, a_2, \dots, a_m \rangle$ and $B = \langle b_1, b_2, \dots, b_n \rangle$, the longest common increasing subsequence (LCIS) problem aims to find an increasing subsequence $C = \langle c_1, c_2, \dots, c_l \rangle$, where $c_1 < c_2 < \dots < c_l$, and l is maximized. Each c_k (for $1 \leq k \leq l$) represents a common element $a_{i_k} = b_{j_k}$, satisfying that $1 \leq i_1 < i_2 < \dots < i_l \leq m$ and $1 \leq j_1 < j_2 < \dots < j_l \leq n$. For instance, $A = \langle 2, 1, 5, 3, 8 \rangle$ and $B = \langle 1, 3, 5, 4, 8, 2 \rangle$, the LCIS of A and B is $\langle 1, 5, 8 \rangle$ or $\langle 1, 3, 8 \rangle$, both of length 3.

Yang *et al.* [20] devised an algorithm for solving the LCIS problem in $O(mn)$ time and space. In their algorithm, $L_j^i[k]$ denotes the smallest ending value of an LCIS of length k between $\langle a_1, a_2, \dots, a_i \rangle$ and $\langle b_1, b_2, \dots, b_j \rangle$. In Table 2.1, since $a_1 = b_6 = 2$, $L_6^1[1] = 1$. For $a_2 = 1$, since $a_2 = b_1 = 1$, $L_1^2[1] = 1$. There is no other match in the second row for $a_2 = 1$, so 1 is assigned to $L_2^2[1]$, $L_3^2[1]$, $L_4^2[1]$, $\dots$, $L_6^2[1]$. Note that, $L_6^2[1]$ is originally set to 2 (from $L_6^1[1] = 2$), but $L_5^2[1] = 1$ is smaller than 2, then, $L_6^2[1] = 2$ is replaced with the better $L_6^2[1] = 1$. Subsequently, we find $a_3 = b_3 = 5$, which can extend the LCIS length from $L_2^2[1] = 1$, hence $L_3^3[2]$ is set to be 5. The entire process is illustrated in Table 2.1.

Table 2.1: An example of the LCIS algorithm proposed by Yang *et al.* [20], where input sequences $A = \langle 2, 1, 5, 3, 8 \rangle$ and $B = \langle 1, 3, 5, 4, 8, 2 \rangle$. The LCIS answer is either $\langle 1, 5, 8 \rangle$ or $\langle 1, 3, 8 \rangle$, both of length 3.

$A \backslash B$		b_1	b_2	b_3	b_4	b_5	b_6
		1	3	5	4	8	2
a_1	2	-	-	-	-	-	$L_6^1[1] = \underline{2}$
a_2	1	$L_1^2[1] = \underline{1}$	$L_2^2[1] = 1$	$L_3^2[1] = 1$	$L_4^2[1] = 1$	$L_5^2[1] = 1$	$L_6^2[1] = 1$
a_3	5	$L_1^3[1] = 1$	$L_2^3[1] = 1$	$L_3^3[1] = 1$ $L_3^3[2] = \underline{5}$	$L_4^3[1] = 1$ $L_4^3[2] = 5$	$L_5^3[1] = 1$ $L_5^3[2] = 5$	$L_6^3[1] = 1$ $L_6^3[2] = 5$
a_4	3	$L_1^4[1] = 1$	$L_2^4[1] = 1$ $L_2^4[2] = \underline{3}$	$L_3^4[1] = 1$ $L_3^4[2] = 3$	$L_4^4[1] = 1$ $L_4^4[2] = 3$	$L_5^4[1] = 1$ $L_5^4[2] = 3$	$L_6^4[1] = 1$ $L_6^4[2] = 3$
a_5	8	$L_1^5[1] = 1$	$L_2^5[1] = 1$ $L_2^5[2] = 3$	$L_3^5[1] = 1$ $L_3^5[2] = 3$	$L_4^5[1] = 1$ $L_4^5[2] = 3$	$L_5^5[1] = 1$ $L_5^5[2] = 3$ $L_5^5[3] = \underline{8}$	$L_6^5[1] = 1$ $L_6^5[2] = 3$ $L_6^5[3] = 8$

2.2 Most Increasing Interval Subsequence

The most increasing interval subsequence (MIIS) problem, proposed by Chen and Yang [5], aims to find the increasing interval subsequence containing the maximum quantity of intervals for the input interval sequence. The quantity of MIIS is similar to the answer length in the LIS problem. The MIIS problem has two variations: comparison by full interval and comparison by a subinterval. Here, we introduce the MIIS variant with comparison by full interval.

Definition 2.1 (interval [5]). $x = [x^s, x^e]$ denotes an interval x which starts from x^s and ends with x^e , where $x^s \leq x^e$.

Definition 2.2 (comparison by full interval [5]). Let $x = [x^s, x^e]$ and $y = [y^s, y^e]$ be two intervals. If $x^e < y^s$, the relation $x <_f y$ holds.

Given an interval sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, the MIIS problem (comparison by full interval) involves removing certain intervals from X , resulting in an interval sequence $C = \langle c_1, c_2, \dots, c_q \rangle$, where $c_1 <_f c_2 <_f \dots <_f c_q$, and the quantity q is maximized. For instance, $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$, the MIIS answer is $\langle [1, 3], [4, 4], [5, 6], [10, 10] \rangle$ or $\langle [2, 3], [4, 4], [5, 6], [10, 10] \rangle$ which has four intervals.

Chen and Yang [5] devised an algorithm that can solve the MIIS problem in $O(n \log n)$ time. In Table 2.2, the recorded values represent the minimal ending value under each q . Upon processing interval $[7, 8]$ after $[1, 3]$, as $x_1 <_f x_2$ ($3 < 7$), q can be extended to 2 with the ending value 8. $x_3 = [2, 3]$ has the same ending value with $x_1 = [1, 3]$, however, it cannot improve the ending value for $q = 1$. When encountering $[7, 8]$ with $[9, 11]$, since $x_2 <_f x_4$ ($8 < 9$), q can be extended to 3 with the ending value 11. When $[4, 4]$ is encountered, since $x_1 <_f x_5$ ($3 < 4$), and $x_5^e < x_2^e$ ($4 < 8$), $[4, 4]$ can provide a better ending value for $q = 2$. Consequently, $[7, 8]$ is replaced by $[4, 4]$, and the ending value for $q = 2$ is updated to 4. Similarly, upon the entry of $[5, 6]$, since $x_5 <_f x_6$ ($4 < 5$), and $x_6^e < x_4^e$ ($6 < 11$), $[5, 6]$ replaces $[9, 11]$. Subsequently, the ending value for $q = 3$ is updated to 6. Finally, since $x_6 <_f x_7$ ($6 < 10$), q can be extended to 4 with the ending value 10.

Table 2.2: An example of the MIIS algorithm (comparison by full interval) proposed by Chen and Yang [5], where the given interval sequence $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$. The MIIS answer can be $\langle [1, 3], [4, 4], [5, 6], [10, 10] \rangle$ or $\langle [2, 3], [4, 4], [5, 6], [10, 10] \rangle$, containing four intervals.

$X \backslash q$		1	2	3	4
x_1	$[1, 3]$	<u>3</u>			
x_2	$[7, 8]$	3	<u>8</u>		
x_3	$[2, 3]$	<u>3</u>	8		
x_4	$[9, 11]$	3	8	<u>11</u>	
x_5	$[4, 4]$	3	<u>4</u>	11	
x_6	$[5, 6]$	3	4	<u>6</u>	
x_7	$[10, 10]$	3	4	6	<u>10</u>

2.3 Longest Increasing Interval Subsequence

The LIIS problem, also introduced by Chen and Yang [5], aims to find an increasing interval subsequence with the maximum union length of intervals. This problem has two variations: comparison by full interval and comparison by a subinterval. In this section, we introduce the LIIS variant with comparison by full interval.

Definition 2.3 (length of an interval [5]). *Let $x = [x^s, x^e]$ be an interval. The length of x , denoted as $|x|$, is defined as $|x| = x^e - x^s + 1$.*

Definition 2.4 (length of an interval sequence [5]). *An interval sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, where each $x_i = [x_i^s, x_i^e]$ for $1 \leq i \leq n$, whose length is denoted as $|X|$. This length is calculated as the sum of the individual lengths of all intervals in X , where any overlapping part is counted only once.*

Given an interval sequence $X = \langle x_1, x_2, \dots, x_n \rangle$, the LIIS problem (comparison by full interval) aims to maximize $|C|$, where $C = \langle c_1, c_2, \dots, c_q \rangle$ is an interval sequence derived from X such that $c_1 <_f c_2 <_f \dots <_f c_q$, achieved by removing certain intervals from X . For example, given $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$, the answer is $\langle [1, 3], [7, 8], [9, 11] \rangle$ with interval sequence length 8.

Chen and Yang [5] designed an algorithm for solving the LIIS problem in $O(n \log n)$ time. In Table 2.3, each pair of values within the parentheses represents two attributes (l, v) : the first attribute l indicates the length of the current increasing interval sequence, and the second attribute v denotes the ending value of the final interval in this interval sequence.

When $x_2 = [7, 8]$ encounters $x_1 = [1, 3]$, since $x_1 <_f x_2$ ($3 < 7$), l can be updated to 5 ($3 + 2$ for interval-union $[1, 3] \cup [7, 8]$), and the ending value of the sequence is now 8. However, since $[1, 3]$ may lead to a better solution later, it must be retained. When $x_3 = [2, 3]$ is encountered, it can generate $(l, v) = (2, 3)$. Since this pair has a shorter length compared to other pairs with the same ending value v generated by x_1 , $(2, 3)$ is subsequently discarded. When $x_4 = [9, 11]$ is encountered, since $x_2 <_f x_4$ ($8 < 9$), l is updated to 8 ($5 + 3$ for intervals $\{[1, 3], [7, 8], [9, 11]\}$), and the ending value of the sequence is now 11. When $x_5 = [4, 4]$ is encountered, it can be appended to the sequence with an ending value 3. Therefore, l becomes 4 ($3 + 1$ for intervals $\{[1, 3], [4, 4]\}$), and the ending value of the sequence becomes 4.

Upon encountering $x_6 = [5, 6]$, it can be appended after the sequence with an ending value 4, resulting in l being 6 ($4 + 2$ for intervals $\{[1, 3], [4, 4], [5, 6]\}$) and the ending value being 6. At this point, $(l, v) = (6, 6)$ has a larger l and a smaller ending value compared to $(5, 8)$. Since an interval that can be appended after a sequence ending with 8 must also be able to be appended after a sequence ending with 6. It indicates that $(6, 6)$ has better potential than $(5, 8)$. Therefore, $(5, 8)$ is deleted. Finally, $x_7 = [10, 10]$ can be appended after the sequence with an ending value of 6, resulting in l being 7 ($6 + 1$ for intervals $\{[1, 3], [4, 4], [5, 6], [10, 10]\}$) and the ending value of the sequence being 10, that is, $(7, 10)$ in the bottom row.

Table 2.3: An example of the LIIS algorithm (comparison by full interval) proposed by Chen and Yang [5], where the given interval sequence $X = \langle [1, 3], [7, 8], [2, 3], [9, 11], [4, 4], [5, 6], [10, 10] \rangle$. The answer is $\langle [1, 3], [7, 8], [9, 11] \rangle$ with interval sequence length 8.

X		$(l, \text{ending value of the sequence})$				
x_1	$[1, 3]$	<u>(3, 3)</u>				
x_2	$[7, 8]$	(3, 3)	<u>(5, 8)</u>			
x_3	$[2, 3]$	(3, 3)	(5, 8)			
x_4	$[9, 11]$	(3, 3)	(5, 8)	<u>(8, 11)</u>		
x_5	$[4, 4]$	(3, 3)	<u>(4, 4)</u>	(5, 8)	(8, 11)	
x_6	$[5, 6]$	(3, 3)	(4, 4)	<u>(6, 6)</u>	(5, 8)	(8, 11)
x_7	$[10, 10]$	(3, 3)	(4, 4)	(6, 6)	<u>(7, 10)</u>	(8, 11)

2.4 Most Common Increasing Interval Subsequence

Definition 2.5 (maximal common interval). *Given two intervals $x = [x^s, x^e]$ and $y = [y^s, y^e]$, if there exists an interval $c = [c^s, c^e]$ such that $c^s = \max(x^s, y^s)$, $c^e = \min(x^e, y^e)$ and $c^s \leq c^e$, then c is a maximal common interval of x and y . If $c^s > c^e$, then a maximal common interval does not exist.*

Figure 2.1 shows an example of a maximal common interval $c = [5, 7]$, constructed from $x = [3, 7]$ and $y = [5, 8]$, which is the intersection of x and y .

x		3	4	5	6	7		
y				5	6	7	8	
c				5	6	7		

Figure 2.1: An example of a maximal common interval.

Definition 2.6 (interval comparison [5]). *Let $x = [x^s, x^e]$ and $y = [y^s, y^e]$ be two intervals. If $x^s < y^s$ and $x^e < y^e$, the relation $x < y$ holds.*

Definition 2.7 (increasing interval sequence). *Given an interval sequence $C = \langle c_1, c_2, \dots, c_q \rangle$. If $c_1 < c_2 < \dots < c_q$, then C is an increasing interval sequence.*

By Definition 2.7, given an increasing interval sequence $C = \langle c_1, c_2, \dots, c_q \rangle$, where $c_k = [c_k^s, c_k^e]$ for $1 \leq k \leq q$, C also satisfies $c_1^s < c_2^s < \dots < c_q^s$ and $c_1^e < c_2^e < \dots < c_q^e$.

Definition 2.8 (MCIIS problem). *Given two interval sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, the most common increasing interval subsequence (MCIIS) problem is to find an increasing interval subsequence $C = \langle c_1, c_2, \dots, c_q \rangle$ that achieves the maximum quantity q . Each c_k , $1 \leq k \leq q$, represents the maximal common interval between x_{i_k} and y_{j_k} , under the condition that $1 \leq i_1 < i_2 < \dots < i_q \leq m$ and $1 \leq j_1 < j_2 < \dots < j_q \leq n$.*

Table 2.4 presents an aligned example of the MCIIS problem with two given interval sequences $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The bottom row displays the maximal common intervals found between X and Y . The MCIIS answer is $\langle [3, 6], [5, 7], [10, 11] \rangle$, consisting of 3 intervals.

In Table 2.4, intervals $x_1 = [1, 6]$ and $y_1 = [3, 11]$ yield the maximal common interval $[3, 6]$ ($\max(1, 3) = 3$, $\min(6, 11) = 6$, for $[3, 6] = [1, 6] \cap [3, 11]$). Similarly, $x_3 = [4, 12]$ and $y_4 = [5, 7]$ result in the maximal common interval $[5, 7]$ ($\max(4, 5) = 5$, $\min(12, 7) = 7$, for $[5, 7] = [4, 12] \cap [5, 7]$), and $x_4 = [10, 11]$ and $y_5 = [3, 14]$ produce the maximal common interval $[10, 11]$ ($\max(10, 3) = 10$, $\min(11, 14) = 11$, for $[10, 11] = [10, 11] \cap [3, 14]$). Furthermore, $\langle [3, 6], [5, 7], [10, 11] \rangle$ is an increasing interval sequence. Among the common increasing interval subsequences found between X and Y , the maximum quantity of intervals is 3. Therefore, $\langle [3, 6], [5, 7], [10, 11] \rangle$ is the MCIIS answer of this example. Note that there may exist multiple MCIIS answers with the maximum quantity of intervals.

Table 2.4: An aligned example of the MCIIS problem, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The MCIIS answer is $\langle [3, 6], [5, 7], [10, 11] \rangle$ with three intervals.

X	[1, 6]	[12, 14]	-	-	[4, 12]	[10, 11]
Y	[3, 11]	-	[6, 9]	[8, 17]	[5, 7]	[3, 14]
max. common interval	[3, 6]	-	-	-	[5, 7]	[10, 11]

2.5 Longest Common Increasing Interval Subsequence

Definition 2.9 (LCIIS problem). *Given two interval sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, the longest common increasing interval subsequence (LCIIS) problem is to find an increasing interval subsequence $C = \langle c_1, c_2, \dots, c_q \rangle$ such that $l = |\cup \{c_k \in C\}|$ is maximized. Each c_k , $1 \leq k \leq q$, represents the maximal common interval between x_{i_k} and y_{j_k} , where $1 \leq i_1 < i_2 < \dots < i_q \leq m$ and $1 \leq j_1 < j_2 < \dots < j_q \leq n$.*

Table 2.5 presents an aligned example of the LCIIS problem with the given interval sequences $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The bottom row displays the length of the maximal common intervals found between X and Y . The LCIIS answer is $\langle [3, 6], [4, 12] \rangle$, with a total length of 10 for $[3, 12] = [3, 6] \cup [4, 12]$. Note that overlapping portions are counted only once. The length of this sequence is 10, $(12 - 3 + 1) = 10$ for $[3, 12] = [3, 6] \cup [4, 12]$, or $(4 + 9 - 3) = 10$, where 3 is the length of overlapping part from 4 to 6.

Table 2.5: An aligned example of the LCIIS problem, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The LCIIS answer is $\langle [3, 6], [4, 12] \rangle$.

X	[1, 6]	[12, 14]	-	-	-	[4, 12]	[10, 11]
Y	[3, 11]	-	[6, 9]	[8, 17]	[5, 7]	[3, 14]	-
max. common interval	[3, 6]	-	-	-	-	[4, 12]	-
interval length	4	-	-	-	-	9	-

CHAPTER 3

Our Algorithms

3.1 Most Common Increasing Interval Subsequence

The MCIIS problem is an extension of the LCIS problem [20]. In the LCIS problem, we seek the longest strictly increasing subsequence that is common to both input sequences. In the MCIIS problem, each element in the sequences is replaced by an interval. As a result, the common parts found are no longer specific values but rather the overlapped segments of two intervals, each from its respective interval sequence.

The primary objective of the MCIIS problem is to identify the maximal common intervals between two interval sequences, where these intervals can generate the increasing interval sequence with the maximum quantity of intervals. In the following, we employ the dynamic programming (DP) approach to track each potential in a cell to evolve into the solution at any given point.

Definition 3.1 (domination of intervals [5]). *Given two intervals $x = [x^s, x^e]$ and $y = [y^s, y^e]$, if $x^s \leq y^s$ and $x^e \leq y^e$, we state that x dominates y .*

Table 3.1 illustrates an example of finding the MCIIS for two given interval sequences $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The entries highlighted in red and bold represent the maximal common intervals found. Each entry is represented as a 3-tuple $\langle q, c_{k'}^s, c_{k'}^e \rangle$, denoting $\langle q$ (quantity), the starting value of the maximal common interval $c_{k'}$ corresponding to

this q , the ending value of the maximal common interval $c_{k'}$ corresponding to this q in detail. In each cell, we check whether an entry can extend from the 3-tuple entries in the upper-left cell, and inherit 3-tuple entries from its upper and its left cells.

Definition 3.2 (domination of 3-tuple entries). *Given two 3-tuple entries $\langle q_1, c_1^s, c_1^e \rangle$ and $\langle q_2, c_2^s, c_2^e \rangle$, where $q_1 = q_2 = q'$. If $c_1^s \leq c_2^s$ and $c_1^e \leq c_2^e$, we say that $\langle q_1, c_1^s, c_1^e \rangle$ dominates $\langle q_2, c_2^s, c_2^e \rangle$.*

In the scenario where several entries have the same quantity q' , a 3-tuple entry that dominates other entries must have equal or better developmental potential. Consider an interval $[c_3^s, c_3^e]$ being generated. If $[c_3^s, c_3^e]$ can extend $\langle q', c_2^s, c_2^e \rangle$ to become $\langle q' + 1, c_3^s, c_3^e \rangle$ (i.e., $c_2^s < c_3^s$ and $c_2^e < c_3^e$), then $[c_3^s, c_3^e]$ must also be able to extend $\langle q', c_1^s, c_1^e \rangle$ to $\langle q' + 1, c_3^s, c_3^e \rangle$, because $c_1^s \leq c_2^s < c_3^s$ and $c_1^e \leq c_2^e < c_3^e$.

In row $x_1 = [1, 6]$ of Table 3.1, x_1 overlaps with $y_1 = [3, 11]$, resulting in the maximal common interval $[3, 6]$. We record $\langle 1, 3, 6 \rangle$ in the cell, where 1 represents the quantity of the increasing interval sequence ending with $[3, 6]$. Continuing from this, it is observed that a maximal common interval $[6, 6]$ is discovered between $x_1 = [1, 6]$ and $y_2 = [6, 9]$. In cases where the quantities are equal ($q = 1$), the dominance principle favors $[3, 6]$ over $[6, 6]$. Consequently, only $\langle 1, 3, 6 \rangle$ is retained for further computational purposes. The maximal common interval $[5, 6]$ generated by $x_1 = [1, 6]$ and $y_4 = [5, 7]$ will also be discarded as it is dominated by $[3, 6]$.

In row $x_2 = [12, 14]$, x_2 and $y_3 = [8, 17]$ can generate the maximal common interval $[12, 14]$. Since $[12, 14]$ can extend the interval with quantity 1 from the upper-left cell, $[3, 6]$, it results in $\langle 2, 12, 14 \rangle$. This indicates that there is an increasing interval sequence ending with $[12, 14]$ with a quantity of 2. By inheriting $\langle 1, 3, 6 \rangle$ from the upper and the left cells, we obtain the final result for this cell.

In row $x_3 = [4, 12]$, x_3 and $y_1 = [3, 11]$ can generate $\langle 1, 4, 11 \rangle$, but $\langle 1, 4, 11 \rangle$ is dominated by $\langle 1, 3, 6 \rangle$. x_3 and $y_2 = [6, 9]$ can extend $\langle 2, 6, 9 \rangle$ from $\langle 1, 3, 6 \rangle$ in its upper-left cell. Then x_3 and $y_3 = [8, 17]$ can extend from the upper-left cell's $\langle 1, 3, 6 \rangle$ to $\langle 2, 8, 12 \rangle$. By inheriting from the upper and the left cells, we additionally

have $\langle 1, 3, 6 \rangle$, $\langle 2, 6, 9 \rangle$ and $\langle 2, 12, 14 \rangle$. However, since the quantity $q = 2$ for $[6, 9]$ dominates $[8, 12]$ and $[12, 14]$, only $\langle 1, 3, 6 \rangle$ and $\langle 2, 6, 9 \rangle$ will be retained.

After processing as described above, the maximum quantity q of the MCIIS problem is recorded in the most bottom-right cell of Table 3.1. In this example, the maximum value of q is 3, indicating that the maximal common intervals found between X and Y can form an increasing interval sequence with a maximum quantity of 3. The MCIIS answer $\langle [3, 6], [5, 7], [10, 11] \rangle$ can be obtained through backtracking.

It is noteworthy that when $q = 2$, both $\langle 2, 5, 7 \rangle$ and $\langle 2, 4, 12 \rangle$ coexist, as $[5, 7]$ and $[4, 12]$ could potentially evolve into better solutions. For instance, if a subsequent maximal common interval $[6, 9]$ is identified, it can extend from $[5, 7]$ but not from $[4, 12]$. Similarly, if another maximal common interval $[5, 14]$ is discovered, it can extend from $[4, 12]$ but not from $[5, 7]$. Consequently, both 3-tuple entries must be retained to accurately represent the possible intervals and their potential for further optimization.

Table 3.1: An example of finding MCIIS, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The MCIIS answer is $\langle [3, 6], [5, 7], [10, 11] \rangle$, consisting of 3 intervals.

$X \backslash Y$		y_1	y_2	y_3	y_4	y_5
		$[3, 11]$	$[6, 9]$	$[8, 17]$	$[5, 7]$	$[3, 14]$
x_1	$[1, 6]$	$\langle \mathbf{1}, \mathbf{3}, \mathbf{6} \rangle$	$\langle \mathbf{1}, \mathbf{6}, \mathbf{6} \rangle$ $\langle 1, 3, 6 \rangle$	$\langle 1, 3, 6 \rangle$	$\langle \mathbf{1}, \mathbf{5}, \mathbf{6} \rangle$ $\langle 1, 3, 6 \rangle$	$\langle \mathbf{1}, \mathbf{3}, \mathbf{6} \rangle$
x_2	$[12, 14]$	$\langle 1, 3, 6 \rangle$	$\langle 1, 3, 6 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{2}, \mathbf{12}, \mathbf{14} \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 12, 14 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{2}, \mathbf{12}, \mathbf{14} \rangle$
x_3	$[4, 12]$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{1}, \mathbf{4}, \mathbf{11} \rangle$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{2}, \mathbf{6}, \mathbf{9} \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 6, 9 \rangle$ $\langle \mathbf{2}, \mathbf{8}, \mathbf{12} \rangle$ $\langle 2, 12, 14 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{2}, \mathbf{5}, \mathbf{7} \rangle$ $\langle 2, 6, 9 \rangle$ $\langle 2, 12, 14 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 5, 7 \rangle$ $\langle \mathbf{2}, \mathbf{4}, \mathbf{12} \rangle$ $\langle 2, 12, 14 \rangle$
x_4	$[10, 11]$	$\langle 1, 3, 6 \rangle$ $\langle \mathbf{1}, \mathbf{10}, \mathbf{11} \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 6, 9 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 6, 9 \rangle$ $\langle \mathbf{3}, \mathbf{10}, \mathbf{11} \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 5, 7 \rangle$ $\langle 2, 6, 9 \rangle$ $\langle 3, 10, 11 \rangle$	$\langle 1, 3, 6 \rangle$ $\langle 2, 5, 7 \rangle$ $\langle 2, 4, 12 \rangle$ $\langle \mathbf{3}, \mathbf{10}, \mathbf{11} \rangle$

3.1.1 The Algorithms

For solving the MCIIS problem, Algorithm 1 presents the main procedure, which includes the $\text{EXTEND}(\cdot)$ function for extending entries from the upper-left cell and the $\text{DOMINATE}(\cdot)$ function responsible for eliminating dominated entries. Functions $\text{EXTEND}(\cdot)$ and $\text{DOMINATE}(\cdot)$ are respectively demonstrated in Algorithms 2 and 3.

In Algorithm 1, $D_{i,j}$ records the ordered dominant set within the cell at row i and column j during the DP execution. Line 1 is the initialization step. Since we assume that all starting and ending values of intervals are positive integers, the

maximal common intervals $[c^s, c^e]$ shall also satisfy c^s and c^e being positive integers. By setting $D_{0,0}$, $D_{i,0}$ and $D_{0,j}$ to $\{\langle 0, 0, 0 \rangle\}$ as the boundary conditions in the first row and the first column, it ensures that the maximal common interval can at least extend from $\langle 0, 0, 0 \rangle$ during the execution of $\text{EXTEND}(\cdot)$ in Line 8.

Lines 2 to 8 demonstrate the core of the MCIIS algorithm. Line 4 executes the inheritance from $D_{i-1,j}$ and $D_{i,j-1}$. The maximal common interval generated by the two input interval sequences is calculated in Lines 5 and 6 (refer to Definition 2.5). Lines 7 and 8 illustrate how $D_{i,j}$ is generated when x_i and y_j have a maximal common interval $[c^s, c^e]$. First, $[c^s, c^e]$ searches the upper-left cell by iterating through $D_{i-1,j-1}$ to find the 3-tuple with the maximum extendable quantity. Subsequently, the 3-tuple constructed from $[c^s, c^e]$ undergoes domination with the 3-tuple entries in $D_{i,j}$ (where $D_{i,j}$ has already inherited entries from $D_{i-1,j}$ and $D_{i,j-1}$).

Algorithm 1 The algorithm for solving the MCIIS problem.

Input: Two interval sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$,

where $x_i = [x_i^s, x_i^e]$ and $y_j = [y_j^s, y_j^e]$ for $1 \leq i \leq m$ and $1 \leq j \leq n$.

Output: The maximum quantity q of common intervals included in the answer of MCIIS.

```

1:  $D_{0,0} \leftarrow D_{i,0} \leftarrow D_{0,j} \leftarrow \{\langle 0, 0, 0 \rangle\}$  for  $1 \leq i \leq m, 1 \leq j \leq n$  ▷ initialization
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:      $D_{i,j} \leftarrow \text{DOMINATE}(D_{i-1,j}, D_{i,j-1})$ 
5:      $c^s \leftarrow \max(x_i^s, y_j^s)$ 
6:      $c^e \leftarrow \min(x_i^e, y_j^e)$ 
7:     if  $c^s \leq c^e$  then ▷ maximal common interval  $[c^s, c^e]$  exists
8:        $D_{i,j} \leftarrow \text{DOMINATE}(\text{EXTEND}([c^s, c^e], D_{i-1,j-1}), D_{i,j})$ 
9: return  $\max\{q | \langle q, \cdot, \cdot \rangle \in D_{m,n}\}$ 

```

Algorithm 2 demonstrates how the $\text{EXTEND}(\cdot)$ function operates. Firstly, we initialize the pointer p to the first 3-tuple entry in the set $D_{i-1,j-1}$ that we are

going to iterate through. For an entry $p = \langle p.\text{quantity}, p.\text{start}, p.\text{end} \rangle$, $p.\text{quantity}$ refers to the quantity of this common increasing interval sequence, $p.\text{start}$ and $p.\text{end}$ refer to the starting value and the ending value of the maximal common interval, respectively.

During each iteration, p moves to the next 3-tuple entry in $D_{i-1,j-1}$ until it reaches the last one. When p points to a new entry, the input maximal common interval $[c^s, c^e]$ is compared with the 3-tuple pointed by p . If an extension is feasible (i.e., if the condition in Line 3 is satisfied), then q is updated to $p.\text{quantity} + 1$. Upon executing $\text{EXTEND}(\cdot)$, it signifies the existence of a maximal common interval between x_i and y_j , ensuring the creation of a 3-tuple entry $\langle q, c^s, c^e \rangle$ with quantity $q \geq 1$. Hence, we initialize q as 0, guaranteeing that encountering $\langle 0, 0, 0 \rangle$ in the set can be extended to $\langle 1, c^s, c^e \rangle$.

After iterating through all 3-tuple entries, q is updated to its feasible maximum value. The condition $q \leq p.\text{quantity}$ in Line 3 ensures that the calculation proceeds only when it is possible to improve the result. This optimization avoids unnecessary computations and helps improve the efficiency and accuracy of the algorithm.

Algorithm 2 The algorithm of $\text{EXTEND}([c^s, c^e], D_{i-1,j-1})$, which extends increasing interval sequence from the upper-left cell.

Input: A maximal common interval $[c^s, c^e]$, as well as the dominant set $D' = D_{i-1,j-1}$.

Output: A set containing a 3-tuple entry $\langle q, c^s, c^e \rangle$.

```

1:  $q \leftarrow 0$  ▷ initialization
2: for  $p \leftarrow D'.\text{begin}()$  to  $D'.\text{end}()$  do ▷ pointer  $p$  iterates over 3-tuples in  $D'$ 
3:   if  $q \leq p.\text{quantity}$  and  $c^s > p.\text{start}$  and  $c^e > p.\text{end}$  then
4:      $q \leftarrow p.\text{quantity} + 1$ 
5: return  $\{\langle q, c^s, c^e \rangle\}$ 

```

Algorithm 3 shows the flow of the $\text{DOMINATE}(D'_1, D'_2)$ function. Initially, an empty set D'' is created to store 3-tuple entries, and pointers p_1 and p_2 are initialized to the first 3-tuple entries in D'_1 and D'_2 , respectively. The entries in D'' are sorted in ascending order of q . Intervals with the same q indicate non-dominance and can coexist. The insertion begins with intervals having the maximum starting value (or minimum ending value) and proceeds until all intervals with the same q are processed before advancing to larger q values.

Lines 2 to 15 demonstrate the operation when both sets have not been fully traversed. When p_1 and p_2 point to entries with the same q , if one entry dominates the other, the dominated entry is discarded by advancing its pointer to the next entry in its set. If there is no domination between p_1 and p_2 , the one with a larger starting value and smaller ending value is appended at the end of D'' , and then its pointer is moved to the next. When p_1 and p_2 point to entries with different q , the one with the smaller q is appended to D'' , and its pointer is then moved to the next.

Lines 16 to 19 demonstrate that when one of D'_1 and D'_2 has been fully traversed, the remaining entries in the set that has not been fully traversed yet are sequentially appended to D'' . Since the entries in D'_1 and D'_2 are also ordered according to the rules of D'' , when fetching entries, we start with the most enclosed ones. When q values are the same, only the entry is appended to D'' if its interval entirely contains another, ensuring that D'' is mutually non-dominated and sorted according to the rules. If q values are different, it means that the pointer associated with the larger q has already traversed the entries with the smaller q . To increase q , either the entries with the smaller q have been dominated or successfully appended to D'' . In both cases, the other pointer can still traverse the remaining entries with smaller q into D'' , maintaining the integrity of data structure of D'' .

Algorithm 3 The algorithm of $\text{DOMINATE}(D'_1, D'_2)$, which merges D'_1 and D'_2 and removes the dominated entries.

Input: Two dominant sets D'_1 and D'_2 containing 3-tuple entries.

Output: A dominant set D'' containing 3-tuple entries.

```

1:  $D'' \leftarrow \emptyset$ ,  $p_1 \leftarrow D'_1.\text{begin}()$ ,  $p_2 \leftarrow D'_2.\text{begin}()$  ▷ initialization
2: while  $p_1 \neq D'_1.\text{end}()$  and  $p_2 \neq D'_2.\text{end}()$  do
3:   if  $p_1.\text{quantity} = p_2.\text{quantity}$  then
4:     if  $p_1.\text{start} > p_2.\text{start}$  and  $p_1.\text{end} < p_2.\text{end}$  then
5:        $D''.\text{append}(p_1)$ ,  $p_1 \leftarrow p_1.\text{next}()$ 
6:     else if  $p_1.\text{start} < p_2.\text{start}$  and  $p_1.\text{end} > p_2.\text{end}$  then
7:        $D''.\text{append}(p_2)$ ,  $p_2 \leftarrow p_2.\text{next}()$ 
8:     else if  $p_1.\text{start} \leq p_2.\text{start}$  and  $p_1.\text{end} \leq p_2.\text{end}$  then ▷  $p_1$  dominates  $p_2$ 
9:        $p_2 \leftarrow p_2.\text{next}()$  ▷ discard  $p_2$ 
10:    else ▷  $p_2$  dominates  $p_1$ 
11:       $p_1 \leftarrow p_1.\text{next}()$  ▷ discard  $p_1$ 
12:    else if  $p_1.\text{quantity} < p_2.\text{quantity}$  then
13:       $D''.\text{append}(p_1)$ ,  $p_1 \leftarrow p_1.\text{next}()$ 
14:    else
15:       $D''.\text{append}(p_2)$ ,  $p_2 \leftarrow p_2.\text{next}()$ 
16: while  $p_1 \neq D'_1.\text{end}()$  do
17:    $D''.\text{append}(p_1)$ ,  $p_1 \leftarrow p_1.\text{next}()$ 
18: while  $p_2 \neq D'_2.\text{end}()$  do
19:    $D''.\text{append}(p_2)$ ,  $p_2 \leftarrow p_2.\text{next}()$ 
20: return  $D''$ 

```

3.1.2 Complexity Analysis

Our algorithm solves the MCIS problem in $O(n^4)$ time and space, where $m \leq n$. The $\text{EXTEND}(\cdot)$ operation traverses all entries within a cell in $O(n^2)$ time. The $\text{DOMINATE}(D'_1, D'_2)$ operation merges two sets D'_1 and D'_2 , each containing $O(n^2)$ entries, where pointers in these sets are moving in one direction until traversal is complete, also requiring $O(n^2)$ time.

Theorem 3.1. *Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, where $m \leq n$. In each cell, the number of 3-tuples in the dominant set is at most $m \times n$.*

Proof. The starting and ending values of a maximal common interval are identified by comparing the starting values x_i^s and y_j^s and ending values x_i^e and y_j^e of x_i and y_j , respectively, where $1 \leq i \leq m$ and $1 \leq j \leq n$. Therefore, X and Y can generate at most $m \times n$ maximal common intervals. Furthermore, each maximal common interval is extended only once, from its upper-left cell by extending the feasible 3-tuple with the largest q . Hence, there will be at most $m \times n$ 3-tuples in each cell. $\square$

3.2 Longest Common Increasing Interval Subsequence

The primary objective of the LCIS problem is to identify the maximal common intervals formed between two interval sequences, where these intervals can generate the increasing interval sequence with the maximum length of their union. Here, we still rely on dynamic programming (DP) approach to maintain the feasible entries corresponding to the LCIS.

Definition 3.3 (domination among increasing interval sequences [5]). *Given two increasing interval sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$, where $x_i = [x_i^s, x_i^e]$ and $y_j = [y_j^s, y_j^e]$ for $1 \leq i \leq m$ and $1 \leq j \leq n$. X is considered to dominate Y if both conditions $|X| \geq |Y|$ and $|X| - |Y| \geq x_m^e - y_n^e$ hold.*

Definition 3.4 (combination of two contiguous intervals). *Given two intervals $x = [x^s, x^e]$ and $y = [y^s, y^e]$ such that $x < y$, if $y^s - x^e \leq 1$, the two intervals can combine into one interval $z = [z^s, z^e]$, where $z^s = x^s$ and $z^e = y^e$.*

Figure 3.1 shows an example of combining two contiguous intervals, $x = [3, 7]$ and $y = [5, 8]$. The resulting interval, $z = [3, 8]$, is the union of x and y .

x		3	4	5	6	7		
y				5	6	7	8	
z		3	4	5	6	7	8	

Figure 3.1: An example of the interval combination.

When calculating the LCIIS length, overlapping segments between intervals are only counted once. Therefore, when intervals are contiguous, we can combine the two intervals into a single interval with Definition 3.4, thereby maintaining the correctness of the result while reducing space usage. The proof will be presented in Theorem 3.2.

For solving the LCIIS problem, we will first introduce the 3-tuple algorithm, followed by the 2-tuple algorithm. For a 3-tuple entry $\langle l, c^s, c^e \rangle$ in the 3-tuple algorithm, l denotes the length of a corresponding LCIIS, c^s and c^e denote the starting and ending values of the last maximal common interval in this common increasing interval subsequence determined by the l , respectively. In each cell, we verify its ability to extend entries in the upper-left cell and inherit data from both its upper and left cells.

Table 3.2 shows an example with two given interval sequences $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. In row $x_1 = [1, 6]$, x_1 and $y_1 = [3, 11]$ construct the maximal common interval $[3, 6] = [1, 6] \cap [3, 11]$ with length 4. We record $\langle 4, 3, 6 \rangle$ in this cell, where $l = 4$ represents the length of the increasing interval sequence ending with $[3, 6]$. Additionally, $x_1 = [1, 6]$ and $y_2 = [6, 9]$ generate the maximal common interval $[6, 6] = [1, 6] \cap [6, 9]$ with length

1, but $\langle 1, 6, 6 \rangle$ is dominated by $\langle 4, 3, 6 \rangle$ (since $4 - 1 \geq 6 - 6$). Similarly, x_1 and $y_4 = [5, 7]$ generate $\langle 2, 5, 6 \rangle$ which is also dominated by $\langle 4, 3, 6 \rangle$.

Table 3.2: An example of finding LCIS by using the 3-tuple entry $\langle l, c^s, c^e \rangle$, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The answer is $\langle [3, 6], [4, 12] \rangle$, with a total length of 10.

$X \backslash Y$		y_1	y_2	y_3	y_4	y_5
		$[3, 11]$	$[6, 9]$	$[8, 17]$	$[5, 7]$	$[3, 14]$
x_1	$[1, 6]$	$\langle 4, 3, 6 \rangle$	$\langle 1, 6, 6 \rangle$ $\langle 4, 3, 6 \rangle$	$\langle 4, 3, 6 \rangle$	$\langle 2, 5, 6 \rangle$ $\langle 4, 3, 6 \rangle$	$\langle 4, 3, 6 \rangle$
x_2	$[12, 14]$	$\langle 4, 3, 6 \rangle$	$\langle 4, 3, 6 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 12, 14 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 12, 14 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 12, 14 \rangle$
x_3	$[4, 12]$	$\langle 4, 3, 6 \rangle$ $\langle 8, 4, 11 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 3, 9 \rangle$ $\langle 8, 4, 11 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 3, 9 \rangle$ $\langle 7, 12, 14 \rangle$ $\langle 8, 4, 11 \rangle$ $\langle 9, 8, 12 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 5, 3, 7 \rangle$ $\langle 7, 3, 9 \rangle$ $\langle 7, 12, 14 \rangle$ $\langle 9, 8, 12 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 3, 9 \rangle$ $\langle 7, 12, 14 \rangle$ $\langle 9, 8, 12 \rangle$ $\langle 10, 3, 12 \rangle$
x_4	$[10, 11]$	$\langle 2, 10, 11 \rangle$ $\langle 4, 3, 6 \rangle$ $\langle 8, 4, 11 \rangle$	$\langle 4, 3, 6 \rangle$ $\langle 7, 3, 9 \rangle$ $\langle 8, 4, 11 \rangle$	$\langle 7, 3, 9 \rangle$ $\langle 8, 4, 11 \rangle$ $\langle 9, 3, 11 \rangle$ $\langle 9, 8, 12 \rangle$	$\langle 7, 3, 9 \rangle$ $\langle 9, 3, 11 \rangle$ $\langle 9, 8, 12 \rangle$	$\langle 9, 3, 11 \rangle$ $\langle 10, 3, 12 \rangle$

In row $x_2 = [12, 14]$, x_2 and $y_3 = [8, 17]$ generate the maximal common interval $[12, 14] = [12, 14] \cap [8, 17]$ with length 3, which can extend $\langle 4, 3, 6 \rangle$ in the upper-left cell to become $\langle 7, 12, 14 \rangle$ ($4 + 3 = 7$ for union $[3, 6] \cup [12, 14]$). Because $[3, 6]$ and $[12, 14]$ are not contiguous, $[3, 6]$ may potentially develop further to dominate the result $\langle 7, 12, 14 \rangle$. Therefore, $\langle 4, 3, 6 \rangle$ must be retained.

In row $x_3 = [4, 12]$, x_3 and $y_1 = [3, 11]$ generate the maximal common interval $[4, 11] = [4, 12] \cap [3, 11]$ with length 8, and $\langle 8, 4, 11 \rangle$ can coexist with

$\langle 4, 3, 6 \rangle$. Additionally, x_3 and $y_2 = [6, 9]$ intersect the maximal common interval $[6, 9] = [4, 12] \cap [6, 9]$ with length 4, and it is feasible by extending from $\langle 4, 3, 6 \rangle$ in the upper-left cell. Since $[3, 6]$ and $[6, 9]$ are contiguous, we combine these two intervals into a single interval $[3, 9]$, with length of 7. After inheriting $\langle 4, 3, 6 \rangle$ and $\langle 8, 4, 11 \rangle$ from the upper and the left cells, respectively, we observe that $\langle 4, 3, 6 \rangle$ is dominated by $\langle 7, 3, 9 \rangle$.

Theorem 3.2. *If two intervals can be combined (i.e., they are contiguous), we only retain the 3-tuple entry that represents the resulting interval after combining, and discard the two original intervals that were combined, following the principle of domination.*

Proof. Given two contiguous intervals $c_1 = [c_1^s, c_1^e]$ and $c_2 = [c_2^s, c_2^e]$ such that $c_1 < c_2$, the interval $c_3 = [c_1^s, c_2^e]$ denotes the interval after combining c_1 and c_2 . If there exists an incoming interval $\hat{c} = [\hat{c}^s, \hat{c}^e]$ that can be extended from c_1 , five cases arise:

Case 1: $\hat{c}^s - c_1^e \leq 1$ and $\hat{c}^e \leq c_2^e$ (c_1 and $\hat{c}$ are contiguous)

Suppose the 3-tuple created by c_1 is kept, $\hat{c}$ will generate a 3-tuple $\langle \hat{c}^e - c_1^s + 1, c_1^s, \hat{c}^e \rangle$, but this 3-tuple will finally be dominated by $\langle c_2^e - c_1^s + 1, c_1^s, c_2^e \rangle$ created by c_3 (according to the domination rule $(c_2^e - c_1^s + 1) - (\hat{c}^e - c_1^s + 1) = c_2^e - \hat{c}^e$).

Case 2: $\hat{c}^s - c_1^e > 1$ and $\hat{c}^e \leq c_2^e$ (c_1 and $\hat{c}$ are not contiguous)

Suppose the 3-tuple created by c_1 is kept, $\hat{c}$ will generate a 3-tuple $\langle (\hat{c}^e - \hat{c}^s + 1) + (c_1^e - c_1^s + 1), \hat{c}^s, \hat{c}^e \rangle$, but this 3-tuple will finally be dominated by $\langle c_2^e - c_1^s + 1, c_1^s, c_2^e \rangle$ created by c_3 (according to the domination rule $(c_2^e - c_1^s + 1) - ((\hat{c}^e - \hat{c}^s + 1) + (c_1^e - c_1^s + 1)) > c_2^e - \hat{c}^e$).

Case 3: $\hat{c}^s - c_1^e \leq 1$ and $\hat{c}^e > c_2^e$ (c_1 and $\hat{c}$ are contiguous)

Suppose the 3-tuple created by c_1 is kept, $\hat{c}$ will generate a 3-tuple $\langle \hat{c}^e - c_1^s + 1, c_1^s, \hat{c}^e \rangle$ which will dominate $\langle c_2^e - c_1^s + 1, c_1^s, c_2^e \rangle$ created by c_3 (according to the domination rule $(\hat{c}^e - c_1^s + 1) - (c_2^e - c_1^s + 1) = \hat{c}^e - c_2^e$). In this case, if the 3-tuple of c_1 does not initially exist, $\hat{c}$ can still extend the 3-tuple of c_3 to the correct result (since $c_3 < \hat{c}$).

Case 4: $\hat{c}^s - c_1^e > 1$ and $\hat{c}^s - c_2^e \leq 1$ and $\hat{c}^e > c_2^e$ (c_2 and $\hat{c}$ are contiguous)

Similar to Case 3, $\hat{c}$ will generate the 3-tuple $\langle \hat{c}^e - c_1^s + 1, c_1^s, \hat{c}^e \rangle$ if the 3-tuples of c_1

and c_2 exist. However, even without the presence of these two 3-tuples, we can still extend the 3-tuple of c_3 , $\langle c_2^e - c_1^s + 1, c_1^s, c_2^e \rangle$, to arrive at the correct answer.

Case 5: $\hat{c}^s - c_2^e > 1$ and $\hat{c}^e > c_2^e$ ($\hat{c}$ is not contiguous with any other intervals)
 If the 3-tuples of c_1 and c_2 exist, $\hat{c}$ will generate $\langle (\hat{c}^e - \hat{c}^s + 1) + (c_2^e - c_1^s + 1), \hat{c}^s, \hat{c}^e \rangle$, which is composed of c_1 , c_2 and $\hat{c}$. With the existence of the 3-tuple of c_3 , $\hat{c}$ can reach the correct answer by extending the 3-tuple of c_3 without relying on c_1 and c_2 .

No matter which case occurs, whether the 3-tuples created by c_1 and c_2 exist or not does not affect the development of the LCIIS answer. $\square$

Next, $x_3 = [4, 12]$ and $y_3 = [8, 17]$ share the maximal common interval $[8, 12] = [4, 12] \cap [8, 17]$ with length 5, which can extend $\langle 4, 3, 6 \rangle$ in the upper-left cell to $\langle 9, 8, 12 \rangle$. After inheriting the entries from the upper and the left cells and performing the domination, only $\langle 7, 3, 9 \rangle$ and $\langle 9, 8, 12 \rangle$ are retained. Then, x_3 and $y_5 = [3, 14]$ construct the maximal common interval $[4, 12] = [4, 12] \cap [3, 14]$, which is contiguous with $[3, 6]$ in the upper-left cell. Combining these two intervals, we obtain $\langle 10, 3, 12 \rangle$. After inheriting and performing domination, only $\langle 10, 3, 12 \rangle$ is retained.

Following the operations described above, the result of the LCIIS problem will be recorded in the most bottom-right cell of Table 3.2. In this example, the unique entry $\langle 10, 3, 12 \rangle$ in the bottom rightmost cell holds the maximum length $l = 10$ for all 3-tuple entries, indicating that the maximal common intervals found between X and Y can form an increasing interval sequence with a maximum length of 10. The answer $\langle [3, 6], [4, 12] \rangle$ can be obtained through the backtracking technique.

According to Tables 3.1 and 3.2, note that the answers to the MCIIS and LCIIS problems with the same input interval sequences may differ.

As mentioned above, we observed that, the starting values of intervals were not used except the extension operation. The ability of extension can be determined by deducing the original starting value from the length l and its ending value c^e .

Hence, we can streamline the original 3-tuple entry into a more space-efficient 2-tuple entry.

Definition 3.5 (domination of 2-tuple entries). *Given two 2-tuple entries $\langle l_1, c_1^e \rangle$ and $\langle l_2, c_2^e \rangle$, if $l_1 \geq l_2$ and $l_1 - l_2 \geq c_1^e - c_2^e$, we say that $\langle l_1, c_1^e \rangle$ dominates $\langle l_2, c_2^e \rangle$.*

Theorem 3.3. *Simplifying from 3-tuple entries to 2-tuple entries does not affect the correctness of the answers.*

Proof. When employing 2-tuple entries, since c^s is not recorded, if the found maximal common intervals are not contiguous, we cannot determine the starting value of the last interval in the increasing interval sequence, which may lead to extending an entry that could not be extended. However, due to the existence of domination, these entries that cannot be extended will all be dominated.

Given two maximal common intervals $c_1 = [c_1^s, c_1^e]$ and $c_2 = [c_2^s, c_2^e]$ that are not contiguous, where $c_1 < c_2$, we can construct 2-tuple entries $\langle |c_1|, c_1^e \rangle$ and $\langle |c_1| + |c_2|, c_2^e \rangle$. While $\langle |c_1| + |c_2|, c_2^e \rangle$ cannot yield c_2^s , when a maximal common interval $\hat{c} = [\hat{c}^s, \hat{c}^e]$ that can extend these two 2-tuple entries is encountered, two cases are considered as follows:

Case 1: $\hat{c}^s > c_2^s$

The extension of $\hat{c}$ in the 2-tuple entries follows the same approach as in the 3-tuple entries.

Case 2: $(c_2^e - (|c_1| + |c_2|) + 1) < \hat{c}^s \leq c_2^s$

In this case, $\hat{c}$ should not extend c_2 , but due to the absence of c_2^s in the 2-tuple entry, $\hat{c}$ still extends this 2-tuple entry, resulting in $\langle \hat{c}^e - (c_2^e - (|c_1| + |c_2|) + 1) + 1, \hat{c}^e \rangle$ (due to combination). For the 2-tuple entry resulting from $\hat{c}$ extending c_1 , if c_1 and $\hat{c}$ are contiguous, it results in $\langle \hat{c}^e - c_1^s + 1, \hat{c}^e \rangle$; if they are not contiguous, it results in $\langle |c_1| + (\hat{c}^e - \hat{c}^s + 1), \hat{c}^e \rangle$. Regardless of whether c_1 and $\hat{c}$ are contiguous, the 2-tuple entry resulting from $\hat{c}$ extending c_1 will always dominate the 2-tuple entry resulting from $\hat{c}$ extending c_2 . Therefore, incorrect extensions will be discarded and they do not affect the final result. $\square$

Table 3.3 shows the example of LCIIS finding by the 2-tuple entry $\langle l, c^e \rangle$ for the same input interval sequences that by the 3-tuple entry $\langle l, c^s, c^e \rangle$ in Table 3.2. The LCIIS algorithms using 2-tuple entries will be introduced in Section 3.2.1.

Table 3.3: An example of finding LCIIS by using the 2-tuple entry $\langle l, c^e \rangle$, where $X = \langle [1, 6], [12, 14], [4, 12], [10, 11] \rangle$ and $Y = \langle [3, 11], [6, 9], [8, 17], [5, 7], [3, 14] \rangle$. The answer is $\langle [3, 6], [4, 12] \rangle$, with a total length of 10.

$X \backslash Y$		y_1	y_2	y_3	y_4	y_5
		$[3, 11]$	$[6, 9]$	$[8, 17]$	$[5, 7]$	$[3, 14]$
x_1	$[1, 6]$	$\langle 4, 6 \rangle$	$\langle 1, 6 \rangle$ $\langle 4, 6 \rangle$	$\langle 4, 6 \rangle$	$\langle 2, 6 \rangle$ $\langle 4, 6 \rangle$	$\langle 4, 6 \rangle$
x_2	$[12, 14]$	$\langle 4, 6 \rangle$	$\langle 4, 6 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 14 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 14 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 14 \rangle$
x_3	$[4, 12]$	$\langle 4, 6 \rangle$ $\langle 8, 11 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 9 \rangle$ $\langle 8, 11 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 9 \rangle$ $\langle 7, 14 \rangle$ $\langle 8, 11 \rangle$ $\langle 9, 12 \rangle$	$\langle 4, 6 \rangle$ $\langle 5, 7 \rangle$ $\langle 7, 9 \rangle$ $\langle 7, 14 \rangle$ $\langle 9, 12 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 9 \rangle$ $\langle 7, 14 \rangle$ $\langle 9, 12 \rangle$ $\langle 10, 12 \rangle$
x_4	$[10, 11]$	$\langle 2, 11 \rangle$ $\langle 4, 6 \rangle$ $\langle 8, 11 \rangle$	$\langle 4, 6 \rangle$ $\langle 7, 9 \rangle$ $\langle 8, 11 \rangle$	$\langle 7, 9 \rangle$ $\langle 8, 11 \rangle$ $\langle 9, 12 \rangle$ $\langle 9, 11 \rangle$	$\langle 7, 9 \rangle$ $\langle 9, 11 \rangle$ $\langle 9, 12 \rangle$	$\langle 9, 11 \rangle$ $\langle 10, 12 \rangle$

3.2.1 The Algorithms

Similar to the algorithms for the MCIIS problem, Algorithm 4 presents the main procedure, incorporating the $\text{EXTEND}(\cdot)$ function to extend entries from the upper-left cell, and the $\text{DOMINATE}(\cdot)$ function to eliminate dominated entries. Functions $\text{EXTEND}(\cdot)$ and $\text{DOMINATE}(\cdot)$ are separately outlined in Algorithms 5 and 6.

Unlike MCIIS algorithms, LCIIS algorithms utilize the 2-tuple entry $\langle l, c^e \rangle$ to store necessary information.

In Algorithm 4, $D_{i,j}$ records the ordered dominant set within the cell at row i and column j during the DP execution. Line 1 denotes the initialization step. Since the identified maximal common interval $[c^s, c^e]$ may have $c^s = 1$, initializing $D_{0,0}$, $D_{i,0}$, and $D_{0,j}$ as $\langle 0, 0 \rangle$ could incorrectly indicate that the interval $[0, 0]$ is contiguous with $[c^s, c^e]$. Therefore, we initialize $D_{0,0}$, $D_{i,0}$, and $D_{0,j}$ as $\langle 0, -1 \rangle$ to avoid this issue. This initialization ensures that the maximal common interval can extend from $\langle 0, -1 \rangle$ during the execution of $\text{EXTEND}(\cdot)$ in Line 8, thereby eliminating the need for special case handling in the first row and first column.

Lines 2 to 8 form the core of the algorithm. First, each cell inherits data from its upper and its left cells. Then, it determines if a maximal common interval exists. If it does, the algorithm identifies the maximum l that can be extended from the 2-tuples in its upper-left cell. The operation is almost identical to Algorithm 1, except that the 2-tuple entries are used here.

Algorithm 4 The algorithm for solving the LCIIS problem.

Input: Two interval sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$,

where $x_i = [x_i^s, x_i^e]$ and $y_j = [y_j^s, y_j^e]$ for $1 \leq i \leq m$ and $1 \leq j \leq n$.

Output: The maximum length l in the LCIIS answer.

```

1:  $D_{0,0} \leftarrow D_{i,0} \leftarrow D_{0,j} \leftarrow \{\langle 0, -1 \rangle\}$  for  $1 \leq i \leq m, 1 \leq j \leq n$  ▷ initialization
2: for  $i \leftarrow 1$  to  $m$  do
3:   for  $j \leftarrow 1$  to  $n$  do
4:      $D_{i,j} \leftarrow \text{DOMINATE}(D_{i-1,j}, D_{i,j-1})$ 
5:      $c^s \leftarrow \max(x_i^s, y_j^s)$ 
6:      $c^e \leftarrow \min(x_i^e, y_j^e)$ 
7:     if  $c^s \leq c^e$  then ▷ maximal common interval  $[c^s, c^e]$  exists
8:        $D_{i,j} \leftarrow \text{DOMINATE}(\text{EXTEND}([c^s, c^e], D_{i-1,j-1}), D_{i,j})$ 
9: return  $\max\{l \mid \langle l, \cdot \rangle \in D_{m,n}\}$ 

```

Algorithm 5 demonstrates the operation of $\text{EXTEND}(\cdot)$. The pointer p points to the first 2-tuple in the set $D_{i-1,j-1}$ and iterates to the end. $p.\text{length}$ refers to the value stored in the first attribute of this 2-tuple (the length of this common increasing interval sequence) and $p.\text{end}$ refers to the value stored in the second attribute of this 2-tuple (the ending value of a maximal common interval). The variable l records the length achieved after extension in each iteration, while l' keeps track of the maximum l that can be extended across all iterations.

In each iteration, the algorithm first checks if the identified maximal common interval can be extended by the interval pointed to by p (Line 3). If the extension is possible, it further determines whether the two intervals are contiguous. If they are, l records the combined interval length; if not, it calculates the lengths of the two separate intervals and sums them. Due to the existence of $\langle 0, -1 \rangle$ in the set (see Algorithm 4), it guarantees that when a maximal common interval exists, it always produces $\langle c^e - c^s + 1, c^e \rangle$. Finally, l' is updated to the maximum l encountered among the data that have been iterated through.

Algorithm 6 demonstrates the process of $\text{DOMINATE}(D'_1, D'_2)$. First, an empty set D'' is used to store 2-tuples, with pointers p_1 and p_2 pointing to the first 2-tuple entries in D'_1 and D'_2 , respectively. D'_1 and D'_2 are both dominant sets. Each entry of D'_1 and D'_2 is inserted into D'' in ascending order of l if it is not dominated by the entry pointed to by the other pointer.

Lines 2 to 15 illustrate the process when neither set has been completely examined. If the 2-tuple entries pointed to by p_1 and p_2 have the same l , the one with the larger ending value shall be eliminated, and its pointer moves to the next one in its set. If the 2-tuple entries pointed to by p_1 and p_2 have different l , the algorithm checks whether the 2-tuple entry with the smaller l might be dominated by the other entry. If it does not, the 2-tuple entry with the smaller l is appended to D'' . Regardless of domination, the pointer to the 2-tuple entry with the smaller l moves to the next one. Lines 16 to 19 illustrate that once either D'_1 or D'_2 has

Algorithm 5 The algorithm of EXTEND($[c^s, c^e], D_{i-1,j-1}$), which extends increasing interval sequence from the upper-left cell.

Input: A maximal common interval $[c^s, c^e]$, as well as the dominant set $D' = D_{i-1,j-1}$.

Output: A set containing a 2-tuple entry $\langle l, c^e \rangle$.

```

1:  $l \leftarrow 0, l' \leftarrow 0$  ▷ initialization
2: for  $p \leftarrow D'.\text{begin}()$  to  $D'.\text{end}()$  do ▷ pointer  $p$  iterates over 2-tuple entries in  $D'$ 
3:   if  $c^s > (p.\text{end} - p.\text{length} + 1)$  and  $c^e > p.\text{end}$  then
4:     if  $c^s - p.\text{end} \leq 1$  then ▷ two intervals are contiguous
5:        $l \leftarrow c^e - (p.\text{end} - p.\text{length} + 1) + 1$ 
6:     else
7:        $l \leftarrow p.\text{length} + (c^e - c^s + 1)$ 
8:     if  $l > l'$  then
9:        $l' \leftarrow l$ 
10: return  $\{\langle l', c^e \rangle\}$ 

```

been completely traversed, the remaining 2-tuple entries from the set that has not yet been fully traversed are sequentially appended to D'' .

When creating a new D'' , since the 2-tuples in D'_1 and D'_2 are sorted in ascending order of l , only when there is no domination relationship between the two 2-tuple entries, the one with the smaller l will be put to D'' . Consequently, the 2-tuple entries in D'' will also maintain the ascending order of l .

3.2.2 Complexity Analysis

Our algorithm solves the LCIS problem in $O(n^3)$ time and space, where each cell contains $O(n)$ entries, given that $m \leq n$.

Algorithm 6 The algorithm of $\text{DOMINATE}(D'_1, D'_2)$, which merges D'_1 and D'_2 and removes the dominated entries.

Input: Two dominant sets D'_1 and D'_2 containing 2-tuple entries.

Output: A dominant set D'' containing 2-tuple entries.

```

1:  $D'' \leftarrow \emptyset, p_1 \leftarrow D'_1.\text{begin}(), p_2 \leftarrow D'_2.\text{begin}()$  ▷ initialization
2: while  $p_1 \neq D'_1.\text{end}()$  and  $p_2 \neq D'_2.\text{end}()$  do
3:   if  $p_1.\text{length} = p_2.\text{length}$  then
4:     if  $p_1.\text{end} \leq p_2.\text{end}$  then ▷  $p_1$  dominates  $p_2$ 
5:        $p_2 \leftarrow p_2.\text{next}()$  ▷ discard  $p_2$ 
6:     else ▷  $p_2$  dominates  $p_1$ 
7:        $p_1 \leftarrow p_1.\text{next}()$  ▷ discard  $p_1$ 
8:   else if  $p_1.\text{length} < p_2.\text{length}$  then
9:     if  $p_2.\text{length} - p_1.\text{length} < p_2.\text{end} - p_1.\text{end}$  then
10:       $D''.\text{append}(p_1)$ 
11:       $p_1 \leftarrow p_1.\text{next}()$ 
12:   else
13:     if  $p_1.\text{length} - p_2.\text{length} < p_1.\text{end} - p_2.\text{end}$  then
14:       $D''.\text{append}(p_2)$ 
15:       $p_2 \leftarrow p_2.\text{next}()$ 
16: while  $p_1 \neq D'_1.\text{end}()$  do
17:    $D''.\text{append}(p_1), p_1 \leftarrow p_1.\text{next}()$ 
18: while  $p_2 \neq D'_2.\text{end}()$  do
19:    $D''.\text{append}(p_2), p_2 \leftarrow p_2.\text{next}()$ 
20: return  $D''$ 

```

Theorem 3.4. *Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$. Each cell contains at most $m + n$ 2-tuple entries.*

Proof. The ending value of a maximal common interval is determined by comparing the ending values x_i^e and y_j^e of x_i and y_j , respectively, where $1 \leq i \leq m$ and $1 \leq j \leq n$. Thus, there are at most $m + n$ distinct ending values for maximal common intervals. Additionally, for 2-tuple entries with the same ending value, only the one with the longest length is retained (refer to Definition 3.3). $\square$

The $\text{EXTEND}(c^s, c^e, D')$ function scans through all entries within D' and takes $O(n)$ time. Function $\text{DOMINATE}(D'_1, D'_2)$ merges two sets D'_1 and D'_2 , each holding $O(n)$ entries sorted by length l and the respective pointer moves in one direction until the end of the traversal, which also takes $O(n)$ time. Therefore, the overall time complexity is given as follows.

Theorem 3.5. *The time complexity of Algorithm 4 is $O(n^3)$, where n is the maximum number of intervals in the two input sequences.*

CHAPTER 4

Experimental Results

4.1 Most Common Increasing Interval Subsequence

When solving the MCIIS problem, since only values need to be compared without considering lengths of intervals, in Table 4.1, we shuffle the numbers ranging from 1 to $(m+n) \times 2$ and assign them arbitrarily to the starting and ending values of the input interval sequences. This method ensures that each value appears exactly once, without duplicates. In contrast, Table 4.2 allows any number from 1 to $(m+n) \times 2$ to be assigned to both starting and ending values, indicating the same number might appear in multiple intervals, resulting in duplicates. The “avg. execution time” column is measured in milliseconds (ms). The “ratio” column represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$, which is used to determine whether the amount of data and execution time are positively correlated.

Table 4.1 presents results after 1000 rounds of execution. The “max. # of entries in a cell” column shows linear growth with increasing n , but remains far below the $O(n^2)$ space as mentioned in Theorem 3.1. By underlining and highlighting the execution time in red, we observe that doubling n approximately results in an eightfold increase in execution time, indicating $O(n^3)$ complexity, which is also significantly lower than the theoretical time complexity $O(n^4)$. Based on these observations, we infer that each cell likely contains $O(n)$ entries.

Table 4.1: The number of entries in a cell and the execution time for the MCIIS algorithm, with non-duplicate starting and ending values. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$.

$m \times n$	max. # of entries in a cell	avg. # of entries in a cell	avg. execution time	ratio
10×10	15	3.22	<u>0.53</u>	607.55
20×20	28	6.03	<u>3.48</u>	693.10
30×30	36	8.97	10.98	735.25
40×40	46	11.83	<u>25.27</u>	749.03
50×50	56	14.78	47.59	776.42
60×60	66	17.71	80.55	791.51
70×70	80	20.79	127.38	799.74
80×80	85	23.73	<u>185.71</u>	817.79

In Table 4.2, also based on 1000 rounds of execution, we observe a slight decrease in both “max. # of entries in a cell” and “avg. # of entries in a cell”. This decrease is likely due to the increased frequency of duplicate values, leading to more instances of domination.

Table 4.2: The number of entries in a cell and the execution time for the MCIIS algorithm, with random starting and ending values. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$.

$m \times n$	max. # of entries in a cell	avg. # of entries in a cell	avg. execution time	ratio
10×10	13	2.91	<u>0.50</u>	582.00
20×20	25	5.64	<u>3.41</u>	661.58
30×30	32	8.40	10.39	727.62
40×40	40	11.15	<u>23.89</u>	746.76
50×50	53	13.88	45.30	766.00
60×60	59	16.77	77.05	783.54
70×70	75	19.65	120.65	798.05
80×80	83	22.48	<u>180.29</u>	798.00

4.2 Longest Common Increasing Interval Subsequence

When solving the LCIIS problem, the density of data distribution might affect the number of entries per cell. Therefore, we introduce a new variable α to control the range of values that can be fetched when the input interval sequences are initially assigned, ranging from 1 to $(m + n) \times 2 \times \alpha$. Tables 4.3 and 4.4 show the results with $\alpha = 10$ and $\alpha = 100$, respectively.

Table 4.3 presents the results obtained from the execution of 1000 random inputs in each size. As n increases, the maximum number of entries in a cell does not show significant growth. We speculate that this is because the number of maximal common intervals also increases with n , which in turn raises the probability of

contiguous intervals appearing, leading to more frequent occurrences of domination. Moreover, the average number of entries in a cell does not change with increasing n , but remains constant.

By underlining and highlighting the execution time in red, it reveals that doubling n leads to approximately a fourfold increase in execution time, suggesting $O(n^2)$ complexity. It is significantly lower than the theoretical time complexity $O(n^3)$. Based on these observations, we conclude that for randomly assigned input interval sequences, the average case complexity is $O(n^2)$, while the worst case complexity is $O(n^3)$.

Table 4.3: The number of entries in a cell and the execution time for the LCIIS algorithm, with $\alpha = 10$. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$.

$m \times n$	max. # of entries in a cell	avg. # of entries in a cell	avg. execution time	ratio
10×10	6	1.75	<u>0.35</u>	500.00
20×20	6	1.89	<u>1.43</u>	528.67
30×30	6	1.90	3.22	531.06
40×40	6	1.84	<u>5.59</u>	526.65
50×50	5	1.81	8.65	523.12
60×60	6	1.78	12.31	520.55
70×70	5	1.77	16.43	527.88
80×80	5	1.76	<u>21.46</u>	524.88

Table 4.4 shows the results when $\alpha = 100$, allowing a tenfold increase in the range of values that can be fetched during the initial random assignment compared to Table 4.3. It indicates that increasing α only slightly raises the average number of entries in a cell. However, the maximum number of entries in a cell shows significant increases in some cases. We speculate that as α increases, the probability of

obtaining non-contiguous maximal common intervals also increases within the same number of executions, leading to more entries that can coexist.

Table 4.4: The number of entries in a cell and the execution time for the LCIIS algorithm, with $\alpha = 100$. Here, time is measured in milliseconds (ms), and ratio represents $(m \times n \times \text{average number of entries in a cell}) / \text{execution time}$.

$m \times n$	max. # of entries in a cell	avg. # of entries in a cell	avg. execution time	ratio
10×10	<u>8</u>	1.83	0.35	522.86
20×20	6	1.90	1.43	531.47
30×30	6	1.96	3.31	532.93
40×40	5	1.84	5.58	527.60
50×50	6	1.95	9.07	537.49
60×60	<u>9</u>	1.90	12.89	530.64
70×70	6	1.88	17.18	536.20
80×80	6	1.79	21.66	528.90

In the experimental results, the aim is to reveal the relationship between the number of entries in a cell and the possible forms of input interval sequences. The experimental results can then be used to evaluate and improve complexity analysis of MCIIS and LCIIS problems. Tables 4.1, 4.2, 4.3 and 4.4 indicate that the average required is lower than the time complexity of Theorems 3.1 and 3.4. Meantime, the tables also suggest that the space complexity may be lower than initially anticipated.

CHAPTER 5

Conclusion

In this thesis, we introduce two novel problems, the most common increasing interval subsequence (MCIIS) problem and the longest common increasing interval subsequence (LCIIS) problem. These variations are derived from the longest common increasing subsequence (LCIS) problem by replacing values with intervals. We also propose efficient algorithms for solving these two problems. The MCIIS problem can be solved in $O(n^4)$ time and space, and the LCIIS problem can be solved in $O(n^3)$ time and space, where n denotes the number of intervals within the input interval sequences. Through space reuse, we further reduce the space complexity to $O(n^3)$ for the MCIIS problem and $O(n^2)$ for the LCIIS problem.

In this thesis, we attempt to constrain the number of entries within a cell in the MCIIS problem to $O(n)$ by utilizing the relationship between starting and ending values. This approach aimed to reduce the overall time complexity to $O(n^3)$, but ultimately, it was unsuccessful. The experimental results indicate that when randomly assigning input interval sequences and conducting multiple rounds of experiments, the execution time is lower than the theoretical time complexity. In the future, we may aim to find lower complexity of MCIIS and LCIIS, leveraging the properties of intervals to develop efficient algorithms.

BIBLIOGRAPHY

BIBLIOGRAPHY

- [1] M. R. Alam and M. S. Rahman, “A divide and conquer approach and a work-optimal parallel algorithm for the LIS problem,” *Information Processing Letters*, Vol. 113, No. 13, pp. 470–476, 2013.
- [2] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” *Journal of molecular biology*, Vol. 215, No. 3, pp. 403–410, 1990.
- [3] R. Bellman, “The theory of dynamic programming,” *Bulletin of the American Mathematical Society*, Vol. 60, No. 6, pp. 503–515, 1954.
- [4] S. Bespamyatnikh and M. Segal, “Enumerating longest increasing subsequences and patience sorting,” *Information Processing Letters*, Vol. 76, No. 1-2, pp. 7–11, 2000.
- [5] G.-T. Chen and C.-B. Yang, “Efficient algorithms for the increasing interval subsequence problems,” *Journal of Information Science and Engineering*, 2024.
- [6] X. Chen, S. Kwong, and M. Li, “A compression algorithm for DNA sequences and its applications in genome comparison,” *Proceedings of the fourth annual international conference on Computational molecular biology*, Lyon, France, p. 107, 2000.
- [7] X. Chen, M. Li, B. Ma, and J. Tromp, “DNACompress: fast and effective DNA sequence compression,” *Bioinformatics*, Vol. 18, No. 12, pp. 1696–1698, 2002.
- [8] M. Crochemore and E. Porat, “Fast computation of a longest increasing subsequence and application,” *Information and Computation*, Vol. 208, No. 9, pp. 1054–1059, 2010.
- [9] C. Faloutsos, M. Ranganathan, and Y. Manolopoulos, “Fast subsequence matching in time-series databases,” *ACM Sigmod Record*, Vol. 23, No. 2, pp. 419–429, 1994.

- [10] T.-c. Fu, F.-l. Chung, R. Luk, and C.-m. Ng, “Stock time series pattern matching: Template-based vs. rule-based approaches,” *Engineering Applications of Artificial Intelligence*, Vol. 20, No. 3, pp. 347–364, 2007.
- [11] S. Grumbach and F. Tahi, “A new challenge for compression algorithms: genetic sequences,” *Information processing & management*, Vol. 30, No. 6, pp. 875–886, 1994.
- [12] J. W. Hunt and T. G. Szymanski, “A fast algorithm for computing longest common subsequences,” *Communications of the ACM*, Vol. 20, pp. 350–353, 1977.
- [13] M. Kutz, G. S. Brodal, K. Kaligosi, and I. Katriel, “Faster algorithms for computing longest common increasing subsequences,” *Journal of Discrete Algorithms*, Vol. 9, No. 4, pp. 314–325, 2011.
- [14] R. Lee, R. Chand, S. Tseng, and Y. T. Tsai, *Introduction to the Design and Analysis of Algorithms: A Strategic Approach*. McGraw-Hill, 2006.
- [15] S.-F. Lo, K.-T. Tseng, C.-B. Yang, and K.-S. Huang, “A diagonal-based algorithm for the longest common increasing subsequence problem,” *Theoretical Computer Science*, Vol. 815, pp. 69–78, 2020.
- [16] W. R. Pearson and D. J. Lipman, “Improved tools for biological sequence comparison,” *Proceedings of the National Academy of Sciences*, Vol. 85, No. 8, pp. 2444–2448, 1988.
- [17] C. Schensted, “Longest increasing and decreasing subsequences,” *Canadian Journal of Mathematics*, Vol. 13, pp. 179–191, 1961.
- [18] P. van Emde Boas, R. Kaas, and E. Zijlstra, “Design and implementation of an efficient priority queue,” *Mathematical Systems Theory*, Vol. 10, No. 1, pp. 99–127, 1976.
- [19] H. Wu, B. Salzberg, G. C. Sharp, S. B. Jiang, H. Shirato, and D. Kaeli, “Subsequence matching on structured time series data,” *Proceedings of the 2005 ACM SIGMOD international Conference on Management of Data*, Baltimore, Maryland, USA, pp. 682–693, 2005.
- [20] I.-H. Yang, C.-P. Huang, and K.-M. Chao, “A fast algorithm for computing a longest common increasing subsequence,” *Information Processing Letters*, Vol. 93, No. 5, pp. 249–253, 2005.